Lab 7: Model Registry, Model Serving, and Model Monitoring

- **Course:** Engineering of Intelligent Models
- **Module:** M4. Operations (Deployment & Monitoring)
- **Focus:** MLflow Model Registry, FastAPI Model Serving, Evidently AI Model Monitoring
- **Branch:** `Lab7`

1. Goal of the Laboratory

Up to this point, our models have been evaluated statically. However, an MLOps pipeline is only as valuable as the predictions it serves and its resilience to change. Over time, the statistical properties of the independent variables (X) can change, leading to a phenomenon known as **Data Drift**. When this occurs, the model's performance inevitably degrades. This, of course, leads to **Model Drift**, where the model's predictions become less accurate over time, necessitating retraining or adjustments.

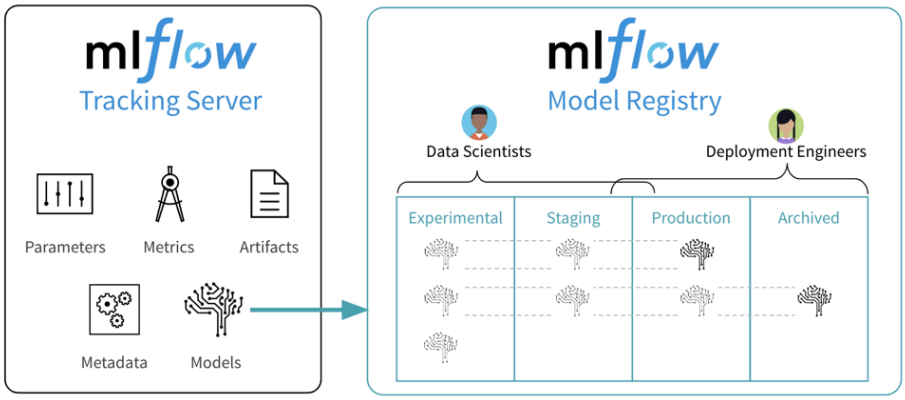
In this laboratory, we will:

1. **Govern our Artifacts:** Use MLflow to formally register our best model and transition it to the `Production` stage.
2. **Serve the Model:** Implement a FastAPI service that dynamically loads the `Production` model from the registry to serve real-time HTTP requests.
3. **Monitor Health:** Integrate Evidently AI as a dedicated service, artificially inject a weather anomaly (Data Drift), and generate a statistical report to trigger our monitoring alarms.

2.1 Model Registry (MLflow Model Registry)

The MLflow Model Registry is a centralized repository for managing the lifecycle of machine learning models. It provides a structured way to track, version, and manage models, making it easier to deploy and maintain them in production. The Model Registry allows us to:

- **Register Models:** Store models with metadata, such as version, stage (e.g., `Staging`, `Production`), and description.
- **Transition Stages:** Move models through different stages of the lifecycle (e.g., from `Staging` to `Production`) to manage their deployment status.
- **Track Lineage:** Keep track of the lineage of models, including the data and code used to create them, which is crucial for reproducibility and auditing.

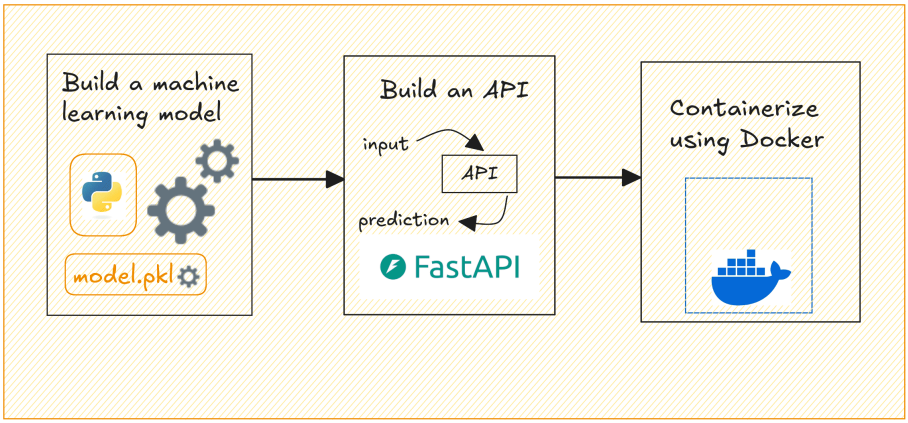


In this Lab, we will use the MLflow Model Registry to register our best-performing model from Lab 6 and transition it to the `Production` stage. This will allow us to serve the model in real-time using FastAPI and monitor its performance over time with Evidently AI.

However, we didn't tag any of our models to `Staging` or `Production` in Lab 6, so for demonstration purposes, we will manually select the best model from the MLflow UI and transition it to `Production`. In a real-world scenario, you would typically automate this process based on performance metrics or other criteria.

2.2 Model Serving (FastAPI)

FastAPI is a modern, fast (high-performance) web framework for building APIs with Python. It is designed to be easy to use and allows for the rapid development of APIs. FastAPI is particularly well-suited for serving machine learning models due to its asynchronous capabilities and support for data validation. With FastAPI, we can create an API endpoint that loads the `Production` model from the MLflow Model Registry and serves predictions in real-time. This allows us to integrate our model into applications and provide predictions to end-users or other systems.



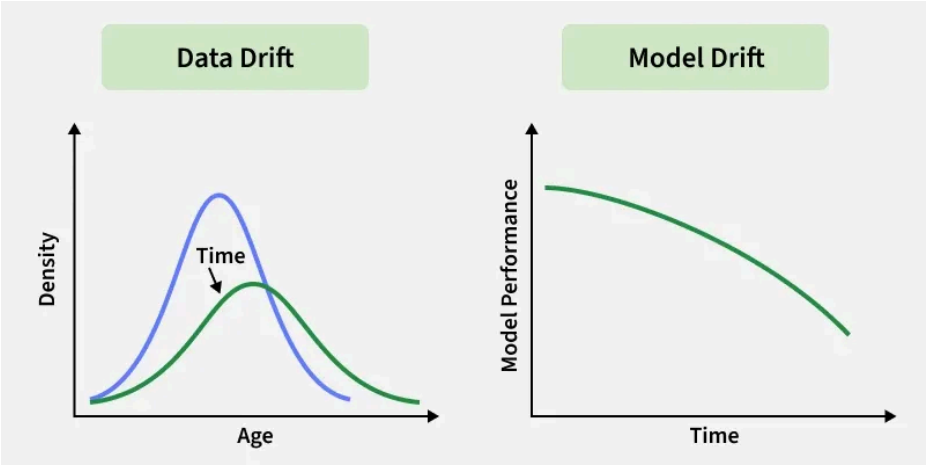
In other words:

- We will save our best model from Lab 6 to the MLflow Model Registry and transition it to `Production`.
- We will implement a FastAPI service that dynamically loads the `Production` model from the registry.
- The FastAPI service will expose an endpoint that accepts input data, processes it, and returns predictions in real-time.

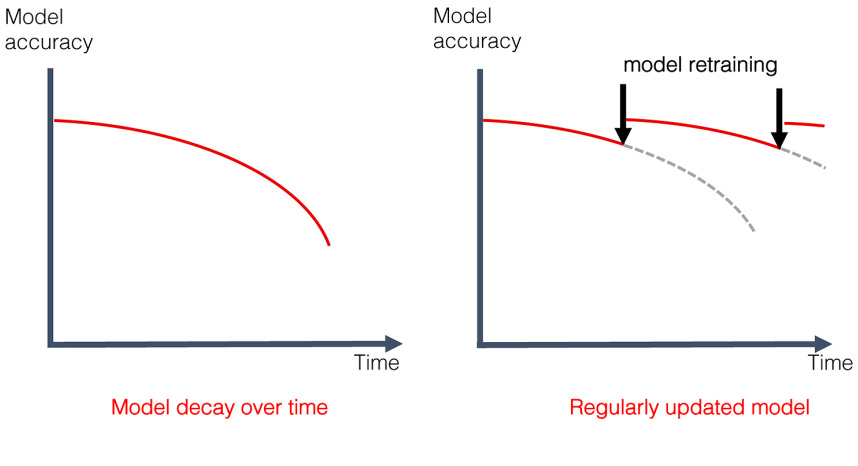
2.3 Data Drift vs. Model Drift

Before we dive into the technical implementation, let's clarify two critical concepts:

- **Data Drift:** Refers to changes in the input data distribution over time. For example, if our model was trained on data collected during normal weather conditions, and suddenly we encounter an extreme weather event (e.g., a hurricane), the statistical properties of the input features may change significantly, leading to degraded model performance.
- **Model Drift:** Refers to the degradation of model performance over time, which can be caused by data drift, changes in the underlying relationships between features and target variable, or even changes in the environment where the model is deployed.



In order to maintain the reliability of our ML models in production, it is crucial to monitor model performance and detect any signs of model drift, since data drift is almost inevitable in real-world applications. By implementing robust monitoring and alerting mechanisms, we can proactively address issues related to data drift and model drift, ensuring that our models continue to deliver accurate predictions over time.



In this lab, we will focus on detecting data drift using Evidently AI, which will help us identify when the input data distribution has changed significantly, allowing us to take appropriate actions such as retraining the model or adjusting our monitoring thresholds.

3. Infrastructure Expansion (Docker Compose)

Before writing Python code, we must expand our infrastructure to include our new microservices: the Inference API and the Monitoring UI.

Step 1: Commit changes and change to Lab7

Save your Lab 6 progress and create an isolated environment for Lab 7:

In [ ]:
```
!git add .
!git commit -m "Lab6: Complete daily evaluation and artifact logging"
!git checkout -b Lab7
```

Step 1: Setup the Directory Structure

In a mature MLOps ecosystem, the environment used to *train* a model must be strictly isolated from the environments used to *serve* and *monitor* it. Monolithic architectures lead to dependency conflicts and security vulnerabilities.

To achieve a production-grade setup, we will create two independent build contexts: one for our FastAPI inference service, and one for our Evidently AI monitoring dashboard. Both will utilize a lightweight Python base image to minimize resource overhead.

Create two new root directories to house the configurations for our new microservices:

In [ ]:
```
# Do not forget to change root folder if you're running this notebook from my official repository
import os
os.chdir('../../') # Change to the root of the repository

# Then, create the necessary directories for the API service
!mkdir api
!mkdir api\app
!mkdir evidently_ui\workspace
```

Step 2: Create the Dedicated API Service

Following standard FastAPI deployment protocols, we create an optimized container for inference.

Create `api/Dockerfile`:

```
# api/Dockerfile
FROM python:3.12-slim

WORKDIR /code
COPY ./api/requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
COPY ./api/app /code/app
```

```
# Start the FastAPI application using the production-ready CLI
CMD ["fastapi", "run", "app/main.py", "--port", "80"]
```

Code Explanation:

- `FROM python:3.12-slim` : We use a lightweight Python image to minimize the container size and reduce attack surface.
- `WORKDIR /code` : Sets the working directory inside the container to `/code` .
- `COPY ./api/requirements.txt /code/requirements.txt` : Copies the API-specific requirements file into the container.
- `RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt` : Installs the necessary dependencies for the API.
- `COPY ./api/app /code/app` : Copies the application code into the container.
- `CMD ["fastapi", "run", "app/main.py", "--port", "80"]` : Starts the FastAPI application on port 80 when the container is run.

Step 3: Create API Requirements

This file should be distinct from your main `requirements.txt` . It only needs the essentials for inference:

```
fastapi[standard]==0.135.2
mlflow==3.10.1
torch==2.10.0
numpy==2.4.3
pandas==2.3.3
```

Step 4: Create the Dedicated Monitoring Service (Evidently AI)

Because Evidently provides a local web server for its dashboard, we will construct a custom Dockerfile that installs the library and initializes its UI on a designated port.

Create `evidently_ui/Dockerfile` :

```
# evidently_ui/Dockerfile
FROM python:3.12-slim

WORKDIR /app
COPY ./evidently_ui/requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt

# Start the Evidently UI server pointing to the mounted workspace
CMD ["evidently", "ui", "--workspace", "/app/workspace", "--host", "0.0.0.0", "--port", "8081"]
```

Step 5: Create Monitoring Requirements

Evidently has its own set of dependencies, so we will create a separate `requirements.txt` for the monitoring service:

```
evidently==0.7.21
```

Step 6: Update Docker Compose

Finally, append these two new independent services to the bottom of your existing `docker-compose.yaml` .

We map port `80` from the API to `8000` on the host, and port `8081` for Evidently.

```
  fastapi-service:
    container_name: emi-fastapi
    build:
      context: .
      dockerfile: api/Dockerfile
    depends_on:
      - mlflow_server
    restart: unless-stopped
    ports:
      - "8000:80"
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow_server:5000
    volumes:
      - ./api/app:/code/app

  evidently-ui:
```

```
    container_name: emi-evidently
    build:
      context: .
      dockerfile: evidently_ui/Dockerfile
    restart: unless-stopped
    ports:
      - "8081:8081"
    volumes:
      - ./evidently_ui/workspace:/app/workspace
```

Step 7: Create a FastAPI Inference Endpoint Placeholder

Before proceeding to the complex PyTorch inference logic, let's create a placeholder script to ensure our decoupled microservice boots and maps the ports correctly.

Create the file `api/app/main.py` and add this basic health-check code:

```
from fastapi import FastAPI

app = FastAPI(title="Weather API", description="Microservice Health Check")

@app.get("/")
def read_root():
    return {"status": "success", "message": "FastAPI Microservice is online and ready for MLflow integration!"}
```

Code Explanation:

- `app = FastAPI(...)` : Initializes a FastAPI application with a title and description.
- `@app.get("/")` : Defines a GET endpoint at the root URL (`/`).
- `def read_root()` : This function is called when the root endpoint is accessed. It returns a JSON response indicating that the FastAPI microservice is online and ready for integration with MLflow. This serves as a simple health check to confirm that our FastAPI service is up and running before we implement the actual inference logic.

Step 8: Rebuild and Restart the Infrastructure

After making these changes, we need to rebuild our Docker images and restart the containers to apply the new configurations

> Note: It will take a few minutes to rebuild the images, especially the FastAPI service due to the additional dependencies. Be patient while Docker processes the new configurations.

In [ ]:
```
!docker compose down
!docker compose up -d --build
```

Now, after you ran this command, you can visit:

- `http://localhost:8000/` to see the FastAPI health check response (it should return a JSON message confirming the service is online).
- `http://localhost:8081/` to access the Evidently AI dashboard (it will be empty for now, but we will populate it with data in the next steps).

4. Model Governance (MLflow Model Registry)

Before a model can be served by our FastAPI microservice, it must be formally governed. In the MLflow paradigm, the Tracking Server (where experiments are logged) is distinct from the Model Registry (where artifacts are versioned and staged).

We will write an automated governance script that programmatically queries the Tracking Server, identifies the exact training run that yielded the lowest `daily_test_rmse`, registers its artifact, and officially promotes it to the `Production` stage.

Step 1: The Governance Script

In your primary development environment (where Airflow and your training scripts reside), create `src/registry/register_model.py`.

```
import mlflow
from mlflow.tracking import MlflowClient
```

```python
import logging

logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')


def promote_best_model(model_name="WeatherForecastModel"):
    """
    Programmatically identifies the best historical run and promotes
    its artifact to the Production stage in the Model Registry.
    """
    mlflow.set_tracking_uri("http://localhost:5000")  # Or, if inside a docker container, use the
appropriate hostname for the Tracking Server
    client = MlflowClient()
    experiment = client.get_experiment_by_name("Weather_Forecasting_Models")

    # 1. Query the Tracking Server for the lowest Root Mean Squared Error
    runs = client.search_runs(
        experiment_ids=[experiment.experiment_id],
        order_by=["metrics.daily_test_rmse ASC"],
        max_results=1
    )

    if not runs:
        logger.error("No tracked runs found. Pipeline must execute training first.")
        return

    best_run = runs[0]
    best_run_id = best_run.info.run_id
    logger.info(f"Optimal run identified: {best_run_id} | RMSE:
{best_run.data.metrics.get('daily_test_rmse'):.4f}")

    # 2. Register the Artifact from the Best Run
    logged_models = best_run.outputs.model_outputs
    last_model = logged_models[0] if len(logged_models) > 0 else None
    model_uri = f"models:/{last_model.model_id}"
    registered_model = mlflow.register_model(
        model_uri=model_uri,
        name=model_name,
        tags={"Run name": best_run.info.run_name, "RMSE": f"
{best_run.data.metrics.get('daily_test_rmse'):.4f}"}
    )

    # 3. Apply an Alias to the Registered Model Version (e.g., "Production")
    client.set_registered_model_alias(name=model_name, alias="Production",
version=registered_model.version)
    logger.info(f"Model '{model_name}' version {registered_model.version} promoted to Production
stage.")


if __name__ == "__main__":
    promote_best_model()
```

Code Explanation:

- `experiment = client.get_experiment_by_name("Weather_Forecasting_Models")` : Retrieves the experiment object by its name to access its runs.
- `runs = client.search_runs(...)` : Queries the Tracking Server for all runs under the specified experiment, and:
  - `experiment_ids=[experiment.experiment_id]` : Filters runs to only those belonging to the specified experiment.
  - `order_by=["metrics.daily_test_rmse ASC"]` : Orders the runs by the `daily_test_rmse` metric in ascending order, ensuring that the best-performing run (with the lowest RMSE) is at the top.
  - `max_results=1` : Limits the results to only the best run.
- `best_run = runs[0]` : Retrieves the best run from the search results.
- `logged_models = best_run.outputs.model_outputs` : Accesses the list of model artifacts logged in the best run.
- `last_model = logged_models[0] if len(logged_models) > 0 else None` : Selects the most recent model artifact from the list (assuming at least one model was logged).
- `model_uri = f"models:/{last_model.model_id}"` : Constructs the URI for the model artifact to be registered in the Model Registry.

- `registered_model = mlflow.register_model(...)` : Registers the model artifact in the Model Registry under the specified name and:
  - `model_uri=artifact_uri` : Specifies the URI of the model artifact to register.
  - `name=model_name` : Names the registered model in the registry.
  - `tags={...}` : Attaches metadata tags to the registered model, including the run name and RMSE for traceability.
- `client.set_registered_model_alias(...)` : Sets an alias for the registered model version where:
  - `name=model_name` : Specifies the name of the registered model to which the alias will be applied.
  - `alias="Production"` : Defines the alias name, indicating that this version of the model is in the Production stage.
  - `version=registered_model.version` : Specifies the version of the registered model to which the alias will point.

Step 2: Execute the Governance Script

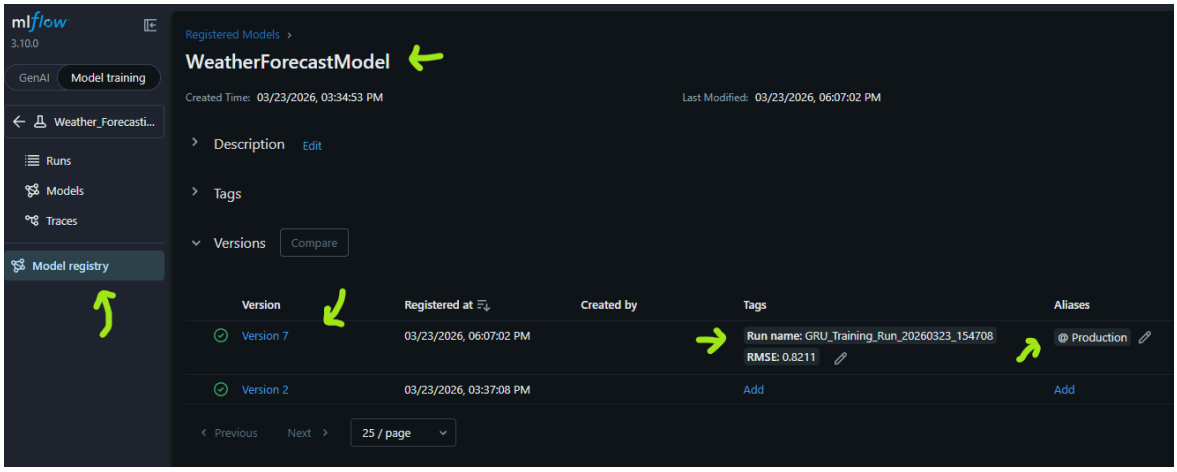
Run this script in your terminal to register the best model and promote it to `Production` :

```
In [ ]: !python src/registry/register_model.py
```

If everything runs successfully, you should see log messages confirming the optimal run ID, its RMSE, and the promotion of the model to the Production stage. Something like this:

```
Optimal run identified: 5f6734507444410daa979d815b67b670 | RMSE: 0.8211
Creted version '1' of model 'WeatherForecastModel'
```

You can also verify this by visiting the MLflow UI and checking the Model Registry section for `WeatherForecastModel` . You should see a new version with the appropriate tags and an alias indicating it is in Production.



> **Note**: In a real-world scenario, you would typically automate this governance process as part of your pipeline, where after each training run, the system evaluates the performance metrics and promotes the best model to Production without manual intervention.

5. Real-Time Model Serving (FastAPI)

With our model safely residing in the `Production` stage, our isolated FastAPI microservice can now consume it.

Notice the architectural elegance of this approach: the API does not hardcode a specific run ID or version number. It requests a dynamic URI (`models:/WeatherForecastModel/Production`). If the Data Science team registers a superior model tomorrow, this API will seamlessly load the new PyTorch weights upon its next instantiation without requiring a single line of application code to be rewritten.

Furthermore, we will design the API endpoint to accept the parameters required by a real-world forecasting application: a target `location` and a future temporal window (`start_date` and `end_date`).

Refactor the API placeholder, and create the application logic inside your decoupled directory: `api/app/main.py` :

```
from fastapi import FastAPI, HTTPException
from mlflow.tracking import MlflowClient
from pydantic import BaseModel, Field
from datetime import date, timedelta
import mlflow
```

```python
import torch
import numpy as np
import pandas as pd
import logging
import os

# Configure isolated logging for the API microservice
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

app = FastAPI(title="Weather Forecasting API", description="MaaS: Model-as-a-Service for PyTorch
inference.")

# 1. State Initialization: Dynamically load the Production Model and its metadata
MLFLOW_TRACKING_URI = os.getenv("MLFLOW_TRACKING_URI")
MODEL_NAME = "WeatherForecastModel"
ALIAS = "Production"

try:
    mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
    client = MlflowClient()

    # 1. Retrieve the model metadata using the Alias
    model_info = client.get_model_version_by_alias(name=MODEL_NAME, alias=ALIAS)
    model_version = model_info.version
    run_id = model_info.run_id

    # 2. Construct the strict versioned URI explicitly (as per docs)
    MODEL_URI = f"models:/{MODEL_NAME}/{model_version}"

    # 3. Load the model
    model = mlflow.pytorch.load_model(MODEL_URI)

    logger.info(f"Loaded {MODEL_NAME} | Alias: {ALIAS} | Version: {model_version} | Run ID:
{run_id}")
except Exception as e:
    logger.error(f"Failed to load model from MLflow Registry: {e}")
    model = None


# 2. Define the exact JSON schema required from the client
class ForecastRequest(BaseModel):
    location: str = Field(..., description="Location of the weather forecast")
    start_date: date = Field(..., description="The future start date for the forecast.")
    end_date: date = Field(..., description="The future end date for the forecast.")


@app.post("/api/v1/forecast")
def generate_forecast(request: ForecastRequest):
    if not model:
        raise HTTPException(status_code=503, detail="Inference model is currently unavailable.")

    if request.start_date > request.end_date:
        raise HTTPException(status_code=400, detail="start_date must precede end_date.")

    try:
        # 1. Read the historical data from the mounted volume
        location = request.location.capitalize()
        try:
            df = pd.read_csv(f"/code/data/raw/historical_weather-{location}.csv")
        except FileNotFoundError:
            raise HTTPException(status_code=500, detail="Location not found.")

        sequence_length = 24  # Must match the sequence length your model was trained on
        feature_cols = ['temperature_2m', 'relative_humidity_2m', 'precipitation']

        # 2. Extract the last N rows of real data
        recent_data = df.tail(sequence_length)[feature_cols].values
        historical_context_tensor = torch.tensor(np.array([recent_data]), dtype=torch.float32)

        # 3. Execute PyTorch Inference
        predictions = []
```

```
        current_date = request.start_date

        with torch.no_grad():
            delta_days = (request.end_date - request.start_date).days + 1

            for _ in range(delta_days):
                # Predict the next time step based on the real historical context
                pred = model(historical_context_tensor)
                predicted_temp = float(pred.numpy()[0][0])

                predictions.append({
                    "date": current_date.isoformat(),
                    "forecasted_temperature_2m": round(predicted_temp, 2)
                })
                current_date += timedelta(days=1)

                # --- The Autoregressive Step ---
                # We feed the predicted temperature back into the model for the next day.
                # Since the model only predicts temperature, we "forward-fill" the last known
                # humidity and precipitation to keep the feature shape consistent.
                last_humidity = float(historical_context_tensor[0, -1, 1])
                last_precip = float(historical_context_tensor[0, -1, 2])

                new_step = torch.tensor([[[predicted_temp, last_humidity, last_precip]]],
dtype=torch.float32)

                # Drop the oldest day (index 0) and append the new predicted day at the end
                historical_context_tensor = torch.cat((historical_context_tensor[:, 1:, :],
new_step), dim=1)

        return {
            "model_version": model_version,  # (Or whatever variable you stored the version in)
            "location": request.location,
            "forecast": predictions
        }

    except Exception as e:
        logger.error(f"Inference error: {str(e)}")
        raise HTTPException(status_code=500, detail="Internal inference execution failed.")
```

Code Explanation:

- `MLFLOW_TRACKING_URI = os.getenv("MLFLOW_TRACKING_URI")` : Retrieves the MLflow Tracking URI from environment variables, allowing for flexible configuration across different environments (e.g., development, staging, production).
- `model_info = client.get_model_version_by_alias(name=MODEL_NAME, alias=ALIAS)` : Fetches the model version information from the MLflow Model Registry using the specified model name and alias. This allows the API to dynamically load the correct model version without hardcoding specific version numbers.
- `MODEL_URI = f"models:/{MODEL_NAME}/{model_version}"` : Constructs the URI for the model artifact in the MLflow Model Registry, which is used to load the model for inference.
- `model = mlflow.pytorch.load_model(MODEL_URI)` : Loads the PyTorch model from the MLflow Model Registry using the constructed URI. This allows the API to serve predictions based on the latest model version that has been promoted to Production.
- `class ForecastRequest(BaseModel)` : Defines a Pydantic model to validate the incoming JSON request body, ensuring that the required fields (`location` , `start_date` , and `end_date`) are present and correctly formatted. This helps to prevent invalid data from reaching the inference logic and provides clear error messages to the client if the input is not as expected.
- `@app.post("/api/v1/forecast")` : Defines a POST endpoint for generating weather forecasts. The endpoint accepts a JSON payload that adheres to the `ForecastRequest` schema, processes the request, and returns a forecast based on the loaded model. The endpoint includes error handling to manage cases where the model is unavailable, the input dates are invalid, or if there are issues during inference execution.
- `def generate_forecast(request: ForecastRequest)` : This function is called when a POST request is made to the `/api/v1/forecast` endpoint.
  - `location = request.location.capitalize()` : Normalizes the location input to match the expected format of the historical data files.
  - `sequence_length = 24` : Defines the number of past time steps the model requires for making a prediction (this should match the sequence length used during training).

- `feature_cols = ['temperature_2m', 'relative_humidity_2m', 'precipitation']` : Specifies the feature columns that the model expects as input.
- `recent_data = df.tail(sequence_length)[feature_cols].values` : Extracts the most recent `sequence_length` rows of data for the specified features to create the input context for the model.
- `historical_context_tensor = torch.tensor(...)` : Converts the extracted data into a PyTorch tensor, which is the required format for the model's input.
- `delta_days = (request.end_date - request.start_date).days + 1` : Calculates the number of days in the requested forecast period.
- `for _ in range(delta_days)` : Iterates over the number of days to generate forecasts for each day in the requested period.
    - `pred = model(historical_context_tensor)` : Executes the model inference to predict the next time step based on the current historical context.
    - `predicted_temp = float(pred.numpy()[0][0])` : Extracts the predicted temperature from the model's output.
    - `predictions.append(...)` : Appends the predicted temperature along with the corresponding date to the list of predictions.
    - `current_date += timedelta(days=1)` : Increments the current date to move to the next day for the forecast.
    - The autoregressive step updates the `historical_context_tensor` by dropping the oldest time step and appending the new predicted day, allowing the model to use its own predictions as input for subsequent forecasts.
    - `last_humidity` and `last_precip` are forward-filled from the last known values to maintain the required input shape for the model, since the model was trained on three features (temperature, humidity, precipitation) but only predicts temperature.
    - `new_step` : Creates a new tensor for the next time step using the predicted temperature and the last known humidity and precipitation values.
    - `historical_context_tensor = torch.cat(...)` : Updates the historical context tensor by concatenating the new predicted step and removing the oldest step, allowing the model to generate forecasts iteratively for the requested date range.
- The function returns a JSON response containing the model version used for inference, the requested location, and the list of forecasted temperatures for the specified date range.

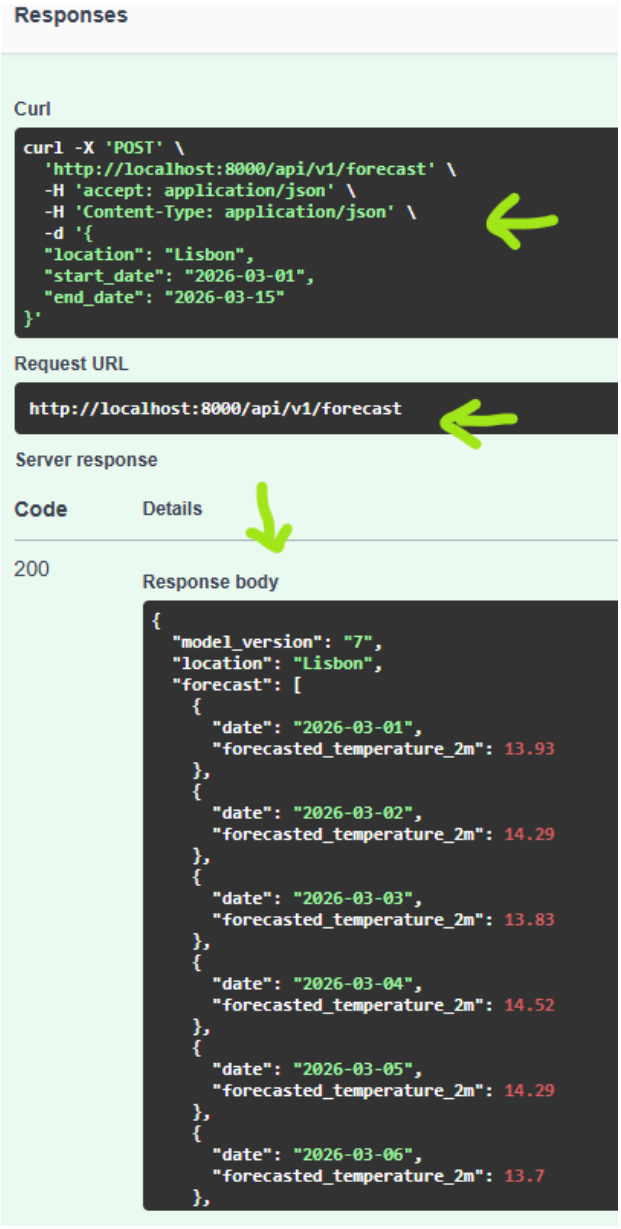
Test the API Endpoint

With the FastAPI service running, you can test the endpoint using `curl`, Postman, or any HTTP client. For this lab, we're using the FastAPI interactive documentation available at `http://localhost:8000/docs`. You can send a POST request to `/api/v1/forecast` with a JSON body like this:

```
{
  "location": "Lisbon",
  "start_date": "2026-03-01",
  "end_date": "2026-03-15"
}
```

By clicking the "Try it out" button, you should be able to see something like this:

This will trigger the inference logic, and you should receive a response containing the forecasted temperatures for the specified date range. The response will also indicate that the model version used, confirming that it is dynamically loading the correct model from the MLflow Model Registry.



6. Provoking and Detecting Data Drift (Evidently AI)

Machine Learning models decay over time. As seasonal patterns shift or climate anomalies occur, the statistical distribution of the live inference data begins to diverge from the historical distribution upon which the model was trained. In academia and industry, this is known as **Covariate Shift** or **Data Drift**. If left undetected, this phenomenon leads to **Model Drift**, where the model's predictions become increasingly inaccurate, necessitating retraining or adjustments.

To validate our monitoring microservice, we will execute an adversarial script that intentionally corrupts the most recent data, generating a statistical proof of drift that our Evidently UI container will display.

We'll also provoke a Model Drift scenario by offsetting the `temperature_2m` target by a significant scalar value, simulating an extreme weather event or sensor malfunction. This will allow us to visualize the impact of data drift on model performance and trigger our monitoring alarms effectively.

In your primary environment, create `src/monitoring/drift_detection.py`. Ensure that this script points to the new `evidently_ui/workspace` directory mapped in your Docker Compose file.

> **Note 1**: The `evidently_ui/workspace` directory is a shared volume between your host machine and the Evidently UI container. This means that any reports generated and saved to this directory by the `drift_detection.py` script will be immediately accessible in the Evidently dashboard without needing to rebuild or restart the container.

> **Note 2**: Also, do not forget to install the `evidently` library in your primary environment to run this script successfully. You can do this by running `pip install evidently` in your terminal and/or adding it to

> your main `requirements.txt` to ensure consistency across your development environment.

```
import pandas as pd
import numpy as np
from evidently import Dataset, Report, DataDefinition, Regression
from evidently.presets import DataDriftPreset, RegressionPreset
from evidently.ui.workspace import Workspace
import logging

# Configure isolated logging for the API microservice
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


def run_drift_analysis():
    logger.info("--- Initiating Covariate Shift & Performance Decay Analysis ---")

    # 1. Load the centralized dataset (Lisbon, for now, to exemplify)
    df = pd.read_csv("data/raw/historical_weather-Lisbon.csv")

    reference_data = df.iloc[:-720].copy()
    current_data = df.iloc[-720:].copy()

    # 2. SIMULATE HISTORICAL PREDICTIONS (Good Performance)
    # The model performed well on the reference data (predictions are very close to the target)
    reference_data['prediction'] = reference_data['temperature_2m'] + np.random.normal(0, 1.5,
size=len(reference_data))

    # 3. PROVOKE DRIFT (The Anomaly)
    # We synthetically offset the actual ground truth temperature readings by +15.0°C
    current_data['temperature_2m'] = current_data['temperature_2m'] + 15.0
    logger.warning("ARTIFICIAL DRIFT INJECTED: Ground truth temperature offset by +15.0°C")

    # 4. SIMULATE RECENT PREDICTIONS (Catastrophic Failure)
    # The model, unaware of the broken sensors/heatwave, predicted normal temperatures.
    # Therefore, its predictions are roughly 15 degrees lower than the new ground truth!
    current_data['prediction'] = current_data['temperature_2m'] - 15.0 + np.random.normal(0, 1.5,
size=len(current_data))
    logger.warning("ARTIFICIAL MODEL DECAY SIMULATED: Predictions are ~15°C lower than the new
ground truth")

    # 5. Define Data Definition
    data_def = DataDefinition(
        numerical_columns=["temperature_2m", "relative_humidity_2m", "precipitation"],
        regression=[Regression(target="temperature_2m", prediction="prediction")]
    )

    # Wrap BOTH raw pandas DataFrames into Evidently Dataset objects
    ref_dataset = Dataset.from_pandas(reference_data, data_definition=data_def)
    curr_dataset = Dataset.from_pandas(current_data, data_definition=data_def)

    # 6. Initialize Workspace
    ws = Workspace.create("evidently_ui/workspace")
    project = ws.create_project("Weather Forecast Operational Monitoring")
    project.description = "Continuous Monitoring for Data Drift and Model Decay."
    project.save()

    # 7. Generate the Combined Report
    drift_report = Report(metrics=[DataDriftPreset(), RegressionPreset()])

    # Pass the Evidenlty Dataset objects
    drift_run = drift_report.run(
        reference_data=ref_dataset,
        current_data=curr_dataset,
        name="Weather Forecast Operational Monitoring"
    )

    # 8. Send to UI
    ws.add_run(project.id, drift_run)
    logger.info("Combined Drift & Performance Report generated! Check http://localhost:8081")
```

```python
if __name__ == "__main__":
    run_drift_analysis()
```

Code Explanation:

- `reference_data` and `current_data`: We split the historical dataset into two parts: the reference data (the older portion) and the current data (the most recent portion). This allows us to compare the two datasets to detect any drift. The reference data simulates the conditions under which the model was trained, while the current data simulates the live environment where drift may occur.
- `reference_data['prediction']`: We simulate the model's predictions on the reference data by adding a small amount of random noise to the actual temperature values. This creates a scenario where the model performs well on the reference data, as the predictions are close to the ground truth.
- `current_data['temperature_2m']`: We intentionally introduce a significant drift by adding a large offset (e.g., +15.0°C) to the actual temperature values in the current data. This simulates a scenario where the input data distribution has changed drastically, which is a common cause of data drift in real-world applications (e.g., due to sensor malfunctions or extreme weather events).
- `current_data['prediction']`: We simulate the model's predictions on the current data by taking the new (drifted) temperature values and subtracting the same offset, along with some random noise. This creates a scenario where the model's predictions are significantly inaccurate compared to the new ground truth, simulating model decay due to data drift.
- `DataDefinition`: We define the structure of our data for Evidently, specifying which columns are numerical and which columns represent the target variable and predictions for regression analysis. This helps Evidently understand how to process the data and calculate the relevant metrics for drift detection and performance evaluation.
- `ref_dataset` and `curr_dataset`: We convert the raw pandas DataFrames into Evidently Dataset objects, which are required for running the drift analysis and generating reports in the Evidently UI.
- `Workspace` and `Project`: We initialize a workspace for Evidently and create a project to organize our monitoring reports. This allows us to manage and visualize the results of our drift analysis in a structured way within the Evidently UI.
- `drift_report.run(...)`: We execute the combined drift and performance report by passing the reference and current datasets. This generates a comprehensive analysis of both data drift and model performance decay, which will be visualized in the Evidently UI.
- `ws.add_run(...)`: We add the generated report to the Evidently workspace, making it accessible through the UI for monitoring and analysis. This allows to easily visualize the impact of data drift on model performance and identify when retraining or adjustments are necessary to maintain the reliability of the model in production.

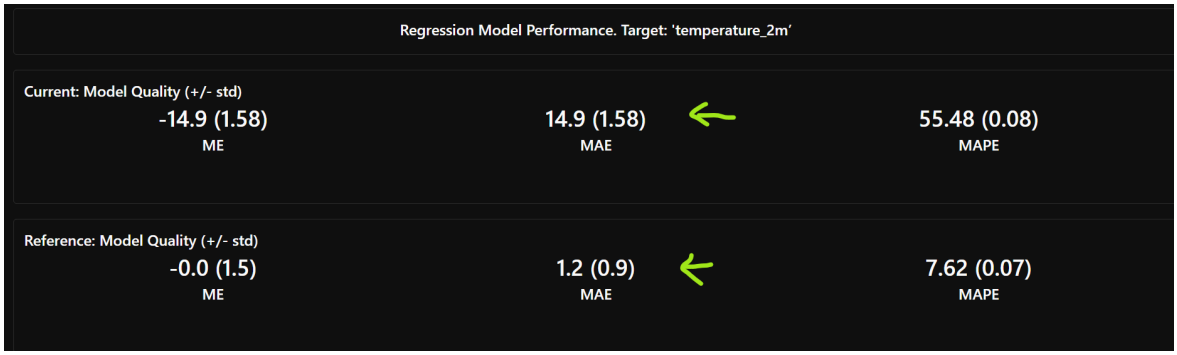
Execute this script:

In [ ]:
```
!python src/monitoring/detect_drift.py
```

Then, navigate to `http://localhost:8081`. You will be greeted by the Evidently AI interface, and by selecting `Projects` then `Reports`, and open the only report generated, will now see **two** major sections in your dashboard:

1. **Data Drift:** Showing the massive statistical shift in the feature distributions.

2. **Regression Performance:** Showing a side-by-side comparison of the ME, MAE, and MAPE. You will clearly see the Reference error at around ~1.5°C, while the Current error will have spiked massively to ~15.0°C, definitively proving that the data anomaly caused your model's accuracy to collapse!



Also, you can check the Error Distribution plot, where the Reference error distribution is tightly clustered around 0°C (indicating good performance), while the Current error distribution is widely spread and centered around -15.0°C, visually confirming the catastrophic model decay caused by the data drift.

7. Conclusion of the Laboratory Series

Congratulations! Over the course of these laboratories, you have successfully engineered a complete, enterprise-grade Machine Learning Operations lifecycle.

You transitioned from isolated Jupyter Notebooks to a robust system featuring:

- **Data Lineage & Orchestration:** DVC and Apache Airflow.
- **Dynamic Configurations:** Hydra.
- **Experiment Tracking & Governance:** MLflow Tracking and Model Registry.
- **Microservices Architecture:** Decoupled PyTorch training and FastAPI inference using optimized Docker builds.
- **Continuous Observability:** Evidently AI for automated drift detection.

In a fully automated corporate environment, the drift alert we just visualized in Evidently would trigger a webhook to our Airflow server, automatically triggering the `daily_weather_ingestion` and `monthly_model_training` DAGs. The system would ingest the newly shifted data, retrain the PyTorch model to adapt to the new reality, register the new weights in MLflow, and the FastAPI service would instantly begin serving the updated intelligence—resulting in a completely self-healing AI architecture.

If you still have the energy, I encourage you to explore that! 😄